Reverse Words in a String

Problem Statement

Given a string `s` containing words separated by one or more dots (`.`), reverse the order of the words.

The output should contain the words in reverse order, separated by a single dot.

Example

```
Input:
s = "i.like.this.program.very.much"

Output:
"much.very.program.this.like.i"
```

Approach

The solution uses the following steps:

1. Split the string into words using `.` as the separator.
2. Traverse the resulting array from **right to left**.
3. Skip empty strings.
4. Add a dot between words.
5. Use `StringBuilder` to construct the final answer.

Java Implementation

```java
class Solution {
    public String reverseWords(String s) {

        String[] arr = s.split("\\.+");

        StringBuilder ans = new StringBuilder();

        for (int i = arr.length - 1; i >= 0; i--) {

            if (arr[i].length() == 0) {
                continue;
            }
```

```
            if (ans.length() > 0) {
                ans.append(".");
            }

            ans.append(arr[i]);
        }

        return ans.toString();
    }
}
```

Code Explanation

1. Split the String

```
String[] arr = s.split("\\.+");
```

This splits the string wherever one or more consecutive dots occur.

Why `\\.+`?

The pattern:

```
\.
```

represents a literal dot.

The `+` means:

```
one or more
```

So:

```
"hello..world...java"
```

is split into:

```
["hello", "world", "java"]
```

This handles both:

```
hello.world
```

and:

```
hello..world
```

---

2. Create StringBuilder

```
StringBuilder ans = new StringBuilder();
```

`StringBuilder` is used to construct the answer efficiently.

Initially:

```
ans = ""
```

---

3. Traverse from Right to Left

```
for (int i = arr.length - 1; i >= 0; i--)
```

Normally, an array is traversed:

```
0 → 1 → 2 → 3
```

Here we traverse in reverse:

```
3 → 2 → 1 → 0
```

This reverses the order of the words.

---

4. Skip Empty Strings

```
if (arr[i].length() == 0) {
    continue;
}
```

If the input contains consecutive dots, an empty string may occur.

For example:

```
"a..b"
```

The empty part between the dots should not appear in the output.

`continue` skips that iteration.

---

5. Add a Dot Between Words

```
if (ans.length() > 0) {
    ans.append(".");
}
```

This prevents an extra dot at the beginning.

For example, while constructing:

```
much
```

we don't add a dot.

Then:

```
much.very
```

Then:

```
much.very.program
```

---

6. Add the Current Word

```
ans.append(arr[i]);
```

The current word is added to the result.

---

7. Return the Result

```
return ans.toString();
```

`StringBuilder` is converted back to a normal `String`.

Dry Run

Consider:

```
s = "i.like.this.program"
```

Step 1: Split

```
arr = ["i", "like", "this", "program"]
```

Step 2: Start from the end

```
i = 3
arr[3] = "program"
```

```
ans = "program"
```

Next

```
i = 2
arr[2] = "this"
```

Since `ans` is not empty, add `.`:

```
ans = "program."
```

Then add `"this"`:

```
ans = "program.this"
```

Next

```
i = 1
arr[1] = "like"
```

```
ans = "program.this.like"
```

Next

```
i = 0
arr[0] = "i"
```

```
ans = "program.this.like.i"
```

Final result:

```
"program.this.like.i"
```

Dry Run with Multiple Dots

Consider:

```
s = "i..like...this.program"
```

After:

```
String[] arr = s.split("\\.+");
```

we get:

```
["i", "like", "this", "program"]
```

The loop starts from:

```
program
```

and produces:

```
program.this.like.i
```

The consecutive dots do not appear in the result.

Iteration Table

For:

```
s = "i.like.this.program"
```

i	arr[i]	ans
3	program	program
2	this	program.this
1	like	program.this.like
0	i	program.this.like.i

Final:

```
program.this.like.i
```

Why Use StringBuilder?

We could concatenate strings using:

```
ans = ans + arr[i];
```

But Java `String` objects are immutable.

Repeated concatenation can create many temporary `String` objects.

`StringBuilder` allows us to modify the same object efficiently:

```
ans.append(arr[i]);
```

Therefore, `StringBuilder` is preferred when repeatedly constructing a string.

Important Parts of the Code

Split

```
s.split("\\.+")
```

Splits on one or more dots.

Reverse traversal

```
for (int i = arr.length - 1; i >= 0; i--)
```

Processes words from right to left.

Avoid extra separator

```
if (ans.length() > 0) {
    ans.append(".");
}
```

Adds `.` only between words.

Add word

```
ans.append(arr[i]);
```

Adds the current word.

Complexity Analysis

Let `n` be the length of the string.

Time Complexity

Splitting the string:

```
O(n)
```

Traversing the words:

```
O(n)
```

Building the result:

```
O(n)
```

Therefore:

```
Time Complexity = O(n)
```

Space Complexity

The split operation creates an array of words and `StringBuilder` stores the result.

Therefore:

```
Space Complexity = O(n)
```

Edge Cases

1. Single Word

```
Input:
"hello"

Output:
"hello"
```

---

2. Two Words

```
Input:
"hello.world"

Output:
"world.hello"
```

---

3. Consecutive Dots

```
Input:
"hello..world"

Output:
"world.hello"
```

---

4. Many Consecutive Dots

```
Input:
"hello...java....world"

Output:
"world.java.hello"
```

DSA Concepts Used

- String manipulation
- Array traversal
- Reverse traversal
- Regular expressions
- `StringBuilder`
- `split()`

Key Takeaway

The main idea is simple:

```
Original:
i → like → this → program

Reverse traversal:
program → this → like → i

Final:
program.this.like.i
```

Pattern

```
String
   ↓
Split into words
   ↓
Traverse from right to left
   ↓
Build result using StringBuilder
```

```
    ↓
Return reversed words
```

Final Complexity

```
Time Complexity  : O(n)
Space Complexity : O(n)
```

Key technique: `split()` + **reverse traversal** + `StringBuilder`.